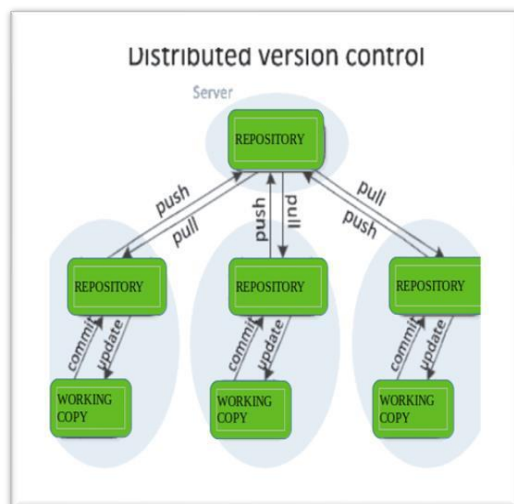




IMS Application in GitHub

bitaiNiazi657 change name of the app and change the widget file in button .		01af6af · 3 days ago	5 commits
android	change name of the app and change the widget file in button .	last wee	
assets	change name of the app and change the widget file in button .	3 days ag	
ios	massain	last wee	
lib	change name of the app and change the widget file in button .	3 days ag	
linux	used launchUrl in onPressed	last wee	
macos	massain	last wee	
test	massain	last wee	
web	massain	last wee	
windows	used launchUrl in onPressed	last wee	
.gitignore	massain	last wee	
.metadata	massain	last wee	
README.md	massain	last wee	
analysis_options.yaml	massain	last wee	
pubspec.lock	used launchUrl in onPressed	last wee	
pubspec.yaml	change name of the app and change the widget file in button .	3 days ag	

CHAPTER 2

2.1 Code Description Flutter container:

A container first surrounds the child with padding (inflated by any borders present in the decoration) and then applies additional constraints to the padded extent (incorporating the width and height as constraints, if either is non-null). The container is then surrounded by additional empty space described from the margin.

During painting, the container first applies the given transform, then paints the decoration to fill the padded extent, then it paints the child, and finally paints the foregroundDecoration, also filling the padded extent.

Containers with no children try to be as big as possible unless the incoming constraints are unbounded, in which case they try to be as small as possible. Containers with children size themselves to their children. The width, height, and constraints arguments to the constructor override this.

By default, containers return false for all hit tests. If the color property is specified, the hit testing is handled by ColoredBox, which always returns true. If the decoration or foregroundDecoration properties are specified, hit testing is handled by Decoration.hitTest.

Container Method

```
(new) Container Container ({
    Key? key,
    AlignmentGeometry? alignment,
    EdgeInsetsGeometry? padding,
    Color? color,
    Decoration? decoration,
    Decoration? foregroundDecoration, double? width, double?
height,
    BoxConstraints? constraints,
    EdgeInsetsGeometry? margin,
    Matrix4? transform,
    AlignmentGeometry? transformAlignment,
    Widget? child,
    Clip clipBehavior = Clip.none,
})
```

2.2 Definition Widget in IMS Application

In computing, a widget is an element of a graphical user interface that displays information or provides a specific way for a user to interact with the operating system (OS) or an application.

IMS Application (SignUpPage.dart) Sample Container and widget Code.

```
Widget textItem(
```

```

String labelText, TextEditingController controller, bool obscureText) {
  return Container(
    width: MediaQuery.of(context).size.width - 70,
    height: 55,
    child: TextFormField(
      controller: controller,
      obscureText: obscureText,
      style: TextStyle(
        fontSize: 17,
        color: Colors.white,
      ),
      decoration: InputDecoration(
        labelText: labelText,
        labelStyle: TextStyle(
          fontSize: 17,
          color: Colors.white,
        ),
        focusedBorder: OutlineInputBorder(
          borderRadius: BorderRadius.circular(15),
          borderSide: BorderSide(
            width: 1.5,
            color: Colors.amber,
          ),
        ),
        enabledBorder: OutlineInputBorder(
          borderRadius: BorderRadius.circular(15),
          borderSide: BorderSide(
            width: 1,
            color: Colors.grey,
          ),
        ),
      ),
    ),
  );
}

```

2.3 Flutter Row and Column

In the previous sections, we have learned to create a simple Flutter application and its basic styling to the widgets. Now, we are going to learn how to arrange the widgets in rows and columns on the screen. The rows and columns are not a single widget; they are two different widgets, namely Row and Column. Here, we will integrate these two widgets together because they have similar properties that help us understand them efficiently and quickly. that allows developers to align children horizontally and vertically according to our needs. These widgets are very necessary when we design the application user interface in Flutter.

2.4 Key Points

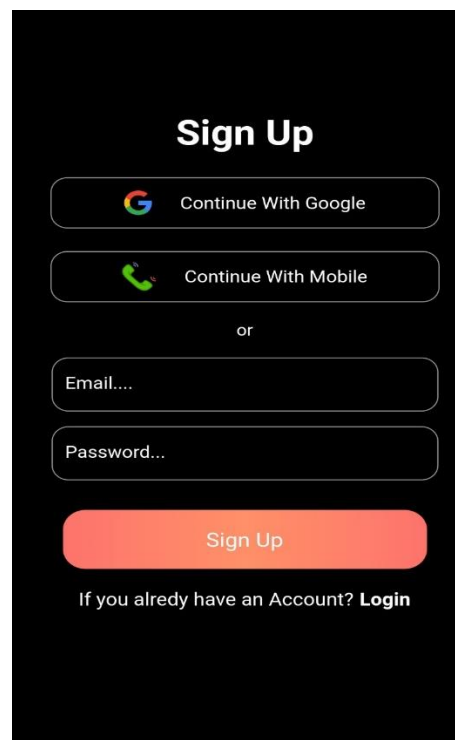
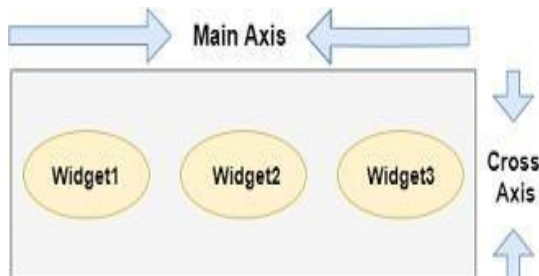
1. Row and Column widgets are the most commonly used layout patterns in the Flutter application.
2. Both may take several child widgets.
3. A child widget can also be a row or column widget.
4. We can stretch or constrain a particular children's widget.
5. Flutter also allows developers to specify how child widgets can use row and column widgets' available space.

2.5 Row Widget

This widget arranges its children in a horizontal direction on the screen. In other words, it will expect child widgets in a horizontal array. If the child widgets need to fill the available horizontal space, we must wrap the children widgets in an Expanded widget.

A row widget does not appear scrollable because it displays the widgets within the visible view. So, it is considered wrong if we have more children in a row which will not fit in the available space. If we want to make a scrollable list of row widgets, we need to use the ListView widget.

We can control how a row widget aligns its children based on our choice using the property **crossAxisAlignment** and **mainAxisAlignment**, the cross-axis will run vertically, and the main axis will run horizontally. see the bellow IMS Application representation to understand it more clearly.



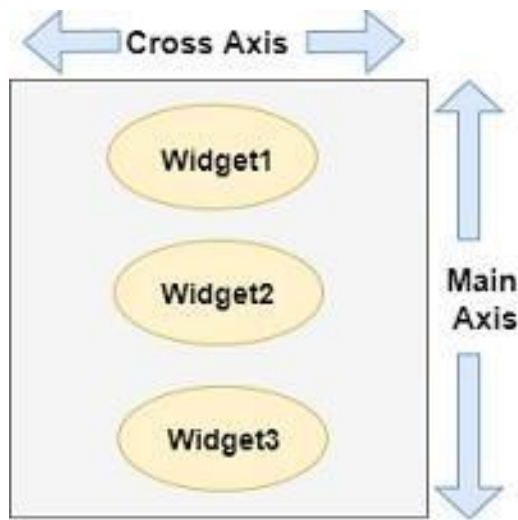
2.6 Column Widget

This widget arranges its children in a vertical direction on the screen. In other words, it will expect a vertical array of children widgets. If the child widgets need to fill the available vertical space, we must wrap the children widgets in an Expanded widget.

A column widget does not appear scrollable because it displays the widgets within the visible view. So, it is considered wrong if we have more children in a column which will not fit in the available space. If we want to make a scrollable list of column widgets, we need to use the ListView Widget.

We can also control how a column widget aligns its children using the property `mainAxisAlignment` and `crossAxisAlignment` the column's cross-axis will run horizontally, and the main axis will run vertically. The below visual representation explains it more clearly.

Column widget also aligns its content by using the same properties as we have discussed in row widget such as `start`, `end`, `center`, `spaceAround`, `spaceBetween`, and `spaceEvenly`. Let us understand it with the help of an example where we are going to align the content such that there is a free space between the children evenly in a column:



2.7 Drawbacks OF Row and Column Widget:

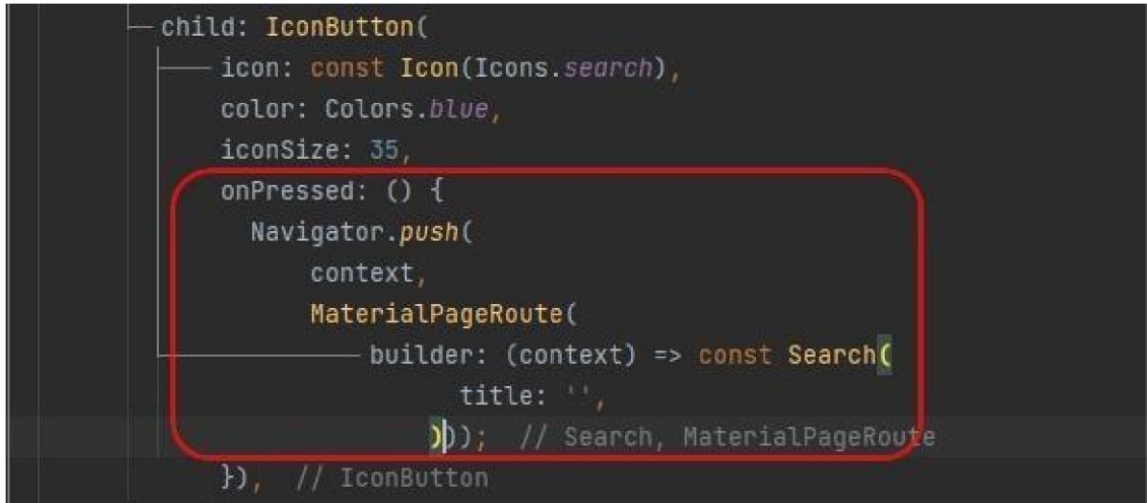
- Row widget in Flutter does not have horizontal scrolling. So if we have inserted a large number of children in a single row that cannot be fit in that row, we will see the Overflow message.
- Column widget in Flutter does not have vertical scrolling. So if we have inserted a large number of children in a single column whose total children size is not equal to the height of the screen, we will see the Overflow message.

2.8 Navigation and routing:

Flutter provides a complete system for navigating between screens and handling deep links. Small applications without complex deep linking can use Navigator, while apps with specific deep linking and navigation requirements should also use the Router to correctly handle deep links on Android and iOS, and to stay in sync with the address bar when the app is running on the web.

2.9 Using the Navigator

The Navigator widget displays screens as a stack using the correct transition animations for the target platform. To navigate to a new screen, access the Navigator through the route's BuildContext and call imperative methods such as `push()` or `pop()`: Because Navigator keeps a stack of Route objects (representing the history stack), The `push()` method also takes a Route object. The `MaterialPageRoute` object is a subclass of Route that specifies the transition animations for Material Design .

A screenshot of a code editor showing a Flutter widget. It's an `IconButton` with a search icon, blue color, and size 35. The `onPressed` callback calls `Navigator.push(context, MaterialPageRoute(builder: (context) => const Search()))`. The `Search` widget is not fully visible. The code is highlighted with a red rounded rectangle.

```
child: IconButton(  
  icon: const Icon(Icons.search),  
  color: Colors.blue,  
  iconSize: 35,  
  onPressed: () {  
    Navigator.push(  
      context,  
      MaterialPageRoute(  
        builder: (context) => const Search(  
          title: '',  
        )), // Search, MaterialPageRoute  
    ), // IconButton  
  },  
)
```

2.10 Drawer to a screen

In apps that use Material Design, there are two primary options for navigation: tabs and drawers. When there is insufficient space to support tabs, drawers provide a handy alternative.

In Flutter, use the Drawer widget in combination with a Scaffold to create a layout with a Material Design drawer. This recipe uses the following steps:

1. Create a Scaffold.
2. Add a drawer.
3. Populate the drawer with items.
4. Close the drawer programmatically.

2.11 Create a Scaffold

To add a drawer to the app, wrap it in a Scaffold widget. The Scaffold widget provides a consistent visual structure to apps that follow the Material Design Guidelines. It also supports special Material Design components, such as Drawers, AppBars, and SnackBars.

In this example, create a Scaffold with a drawer:

```
Scaffold (  
  drawer: // Add a Drawer here in the next step.);
```

2.12 Add a drawer

Now add a drawer to the Scaffold. A drawer can be any widget, but it's often best to use the Drawer widget from the material library, which adheres to the Material Design spec.

```
Scaffold ( drawer: Drawer ( child:
// Populate the Drawer in the next step.
),
);
```

2.13 Populate the drawer with items

Now that you have a Drawer in place, add content to it. For this example, use a ListView. While you could use a Column widget, ListView is handy because it allows users to scroll through the drawer if the content takes more space than the screen supports.

```
Drawer ( child:
ListView (
padding: EdgeInsets.zero,
children: [ const DrawerHeader(
decoration:
BoxDecoration( color:
Colors.blue, ),
child: Text ('Drawer Header'),
),
ListTile (
title: const Text ('Item 1'),
onTap: () { }, ),
ListTile ( title: const Text
('Item 2'), onTap: () { },
), ],
),
```

2.14 Close the drawer programmatically

After a user taps an item, you might want to close the drawer. You can do this by using the Navigator.

When a user opens the drawer, Flutter adds the drawer to the navigation stack. Therefore, to close the drawer, call `Navigator.pop(context)`.

Sample Drawer Code.

```

drawer: Drawer (
  child: ListView (
    // Important: Remove any padding from the ListView.
    padding: EdgeInsets.zero,
    children: [
      const DrawerHeader (
        decoration: BoxDecoration (
          color: Colors.blue,
        ),
        child: Text ('Drawer Header'),
      ),
      ListTile (
        title: const Text ('Item 1'),
        onTap: () {

          Navigator.pop(context);
        },
      ),
      ListTile (
        title: const Text ('Item 2'),
        onTap: () {
          // Update the state of the app
          // ...
          // Then close the drawer
          Navigator.pop(context);
        },
      ),
    ],
  ),
),

```

2.15 Flutter Slider

A slider in Flutter is a material design widget used for selecting a range of values. It is an input widget where we can set a range of values by dragging or pressing on the desired position. In this article, we are going to show how to use the slider widget in Flutter in setting the range of values and how to customize the look of a slider.

Usually, we use the slider widget for changing a value. So, it is required to store the value in a variable. This widget has a slider class that requires the `onChanged ()` function. This function will be called whenever we change the slider position.

A slider can be used for selecting a value from a continuous or discrete set of values. By default, it uses a continuous range of values. If we want to use discrete values, we must have to use a non-null value for divisions. This discrete division indicates the number of discrete intervals. Before getting the value, we have to set the minimum and maximum

value. Slider provides min and max arguments to set the minimum and maximum limitations. For example,

2.16 The slider in Flutter uses the following terms

Thumb: It is a round shape that slides horizontally when we change values by dragging.

Track: It is a horizontal line where we can slide the thumb.

Overlay: It appears around the thumb while dragging.

Tick marks: It is used to mark the discrete values of a slider.

Value indicators: It will show the labels for the thumb values when we defined the labels.

Active: It is the active side of the slider, which is in between the thumb and the minimum value.

Inactive: It is the inactive side of the slider, which is in between the thumb and the maximum value. example Slider...

```
Slider (
  min: 0,
  max: 100,
  value: _value,
  onChanged: (value) {
    setState(() {
      _value = value;
    });
  },
),
```

CHAPTER 3

3.1 Important files

In the important files section, a series of important files are linked, such as Flutter library import at the project of the IMS in pubspec. yaml to support and guide public fertilizers in all important sections and bases but specially dependencies, dev_ dependencies and Flutter Used-materialdesign.

```

# https://developer.apple.com/library/archive/documentation/
version: 1.0.0+1

environment:
  sdk: ">=2.16.2 <3.0.0"

# Dependencies specify other packages that your package needs
# To automatically upgrade your package dependencies to the latest
# consider running `flutter pub upgrade --major-versions`. Alternatively,
# dependencies can be manually updated by changing the version numbers
# the latest version available on pub.dev. To see which dependencies have
# versions available, run `flutter pub outdated`.
dependencies:
  autocomplete_textfield: ^2.0.1
  cloud_firestore: ^3.1.17
  cupertino_icons: ^1.0.2
  email_validator: ^2.0.1
  firebase_auth: ^3.3.18
  firebase_core: ^1.17.0
  firebase_storage: ^10.2.16
  flutter:
    sdk: flutter
  flutter_secure_storage: ^5.0.2
  flutter_svg: ^1.0.3
  flutter_typeahead: ^3.2.4
  google_sign_in: ^5.3.2
  google_sign_in_web: ^0.10.1+2
  intl: ^0.17.0
  otp_text_field: ^1.1.3
  typeahead: ^0.0.1

dev_dependencies:
  flutter_lints: ^1.0.0

```

```

pubspec.yaml
47
48 dev_dependencies:
49   flutter_lints: ^1.0.0
50   flutter_test:
51     sdk: flutter
52
53 # For information on the generic Dart part of this file see the
54 # following page: https://dart.dev/tools/pub/pubspec
55 # The following section is specific to Flutter.
56 flutter:
57
58   # The following line ensures that the Material Icons font is
59   # included with your application, so that you can use the
60   # Material Icons class.
61   uses-material-design: true
62
63   # To add assets to your application, add an assets section to
64   # the file.
65   assets:
66     - assets/
67     - images/a_dot_burr.jpeg
68     - images/a_dot_ham.jpeg
69   # An image asset can refer to one or more resolution-specific
70   # assets via the "resolution" key. For more information, see
71   # https://flutter.dev/assets-and-images/#resolution-aware.
72   # For details regarding adding assets from package dependencies,
73   # see https://flutter.dev/assets-and-images/#from-packages.
74   # To add custom fonts to your application, add a fonts section
75   # to the "flutter" section. Each entry in this list represents a
76   # font family. The "family" key with the font family name, and a "font"
77   # key with the font file name. For example:
78   fonts:
79     - family: MyFlutterApp
80       font: assets/icon/MyFlutterApp.ttf

```

3.2 Database for Mobile App

A database is a way to persist data for an application. A mobile database is a database specifically intended for persisting data in mobile applications, often on the device itself as local storage. This use of local storage is to allow the app to continue to work even when the device is offline or used as a cache for loading data in the event of slow network connectivity .

This often manifests as a file on the device which holds the data in the correct format for whatever database type is being used. However, as we will see later on, there are different types of mobile application databases, and how the information is stored will change depending on what type of database is being used.

The need to store data doesn't change between application types, though. The application might want to store user preferences, profile information, cached data for faster loading times, or any number of different pieces of data. Why this data is stored doesn't change between applications. Instead, the things to consider when choosing a database will be different for mobile apps. We will look further into this later in the article.

3.3 SQLite

SQLite is an embedded, server-less relational database management system. It is an inmemory open-source library with zero configuration and does not require any installation.

Also, it is very convenient as it's less than 500kb in size, which is significantly lesser than other database management systems.



3.4 Why Use SQLite?

- SQLite is an open-source software. The software does not require any license after installation.
- SQLite is serverless as it doesn't need a different server process or system to operate.
- SQLite facilitates you to work on multiple databases on the same session simultaneously, thus making it flexible.
- SQLite is a cross-platform DBMS that can run on all platforms, including macOS, Windows, etc.
- SQLite doesn't require any configuration. It needs no setup or administration.

3.5 When to Use SQLite?

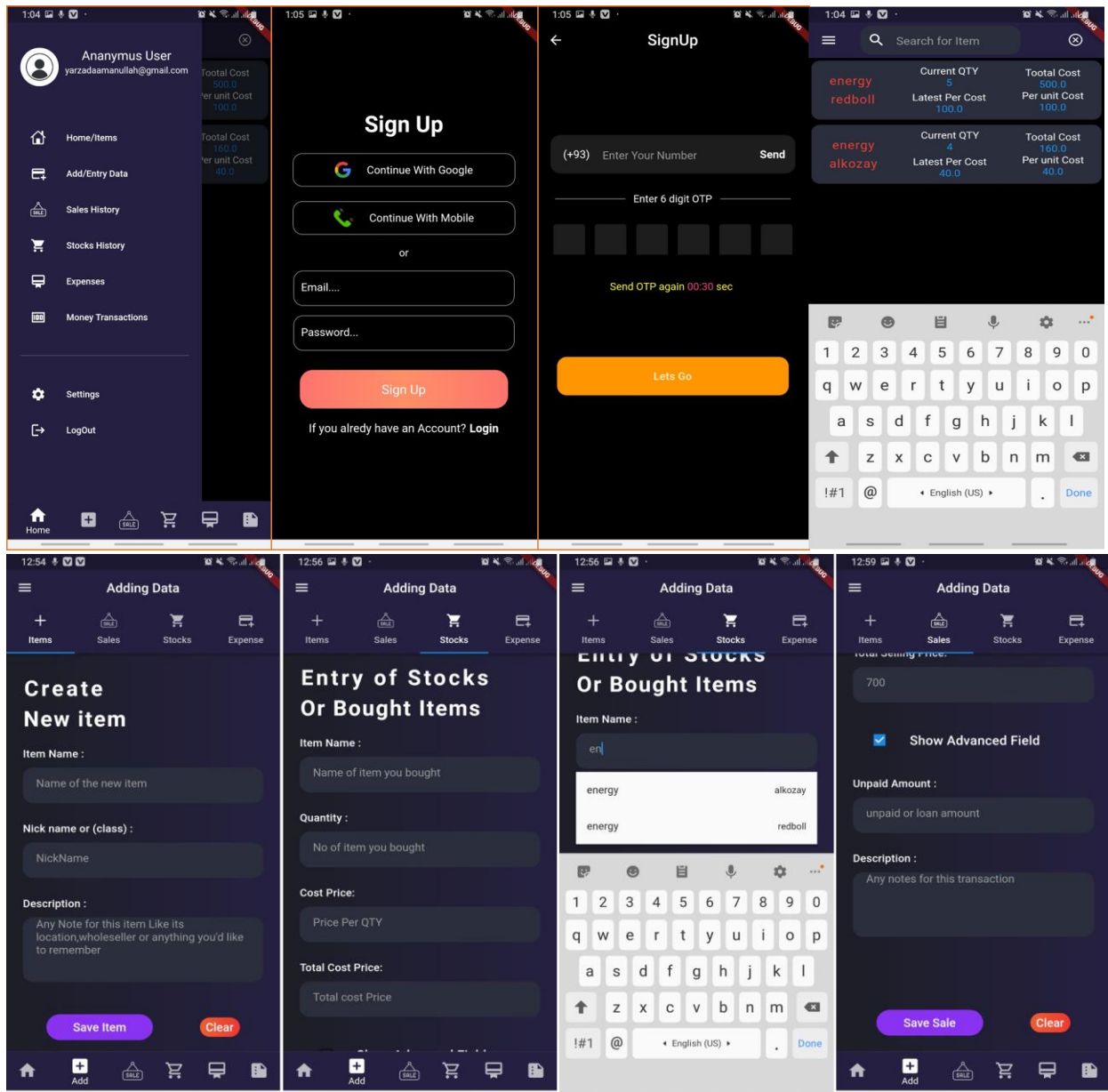
- SQLite is used to develop embedded software for devices like televisions, cell phones, cameras, etc.
- It can manage low to medium-IMS HTTP requests.
- SQLite can change files into smaller size archives with lesser metadata.
- SQLite is used as a temporary dataset to get processed with some data within an application.
- Beginners use SQLite for learning and training purposes, as it requires no installation and configuration.

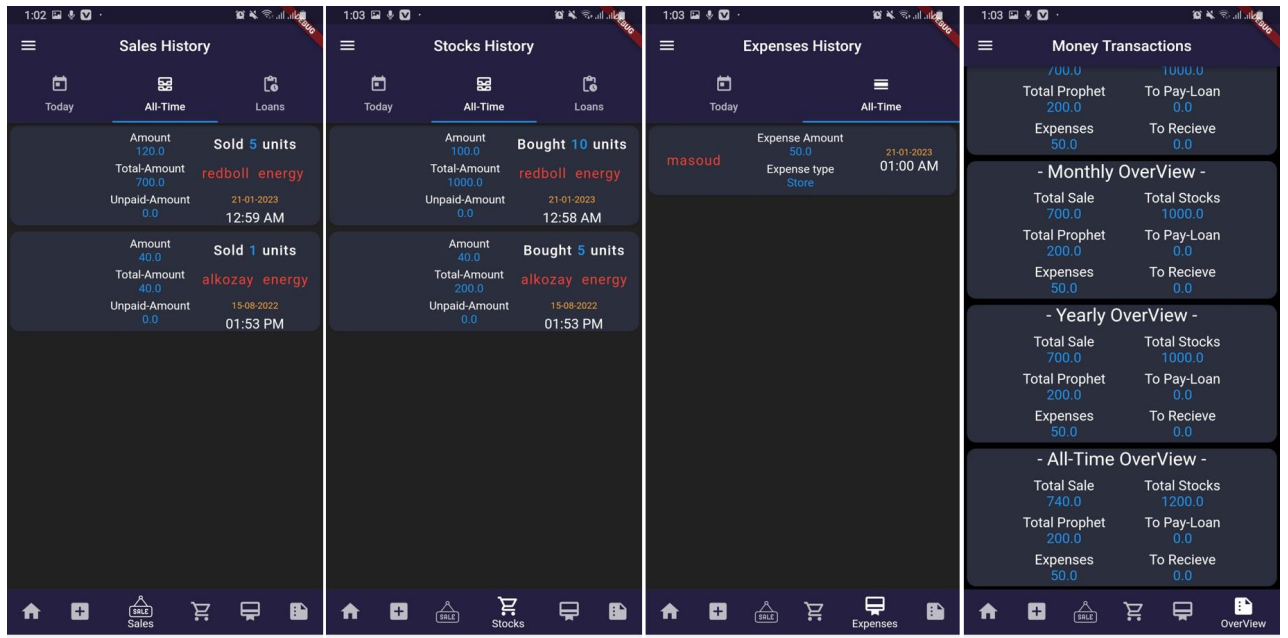
Note : Because our IMS Application is an online service thus we use Firebase Firestore Database in our Project but SQLite is a very common Database in Mobile software development and we wanted have some discussion about it .

CHAPTER 4

4.1 Project screenshot

In this section, for the introduction and better acquaintance of IMS, its important parts are displayed for you, such as SignUp Screen, Drawer, Items Screen, Transaction Screen, data Entry, Sales and stocks History . Expenses, . Etc....





4.2 IMS Application supported platforms

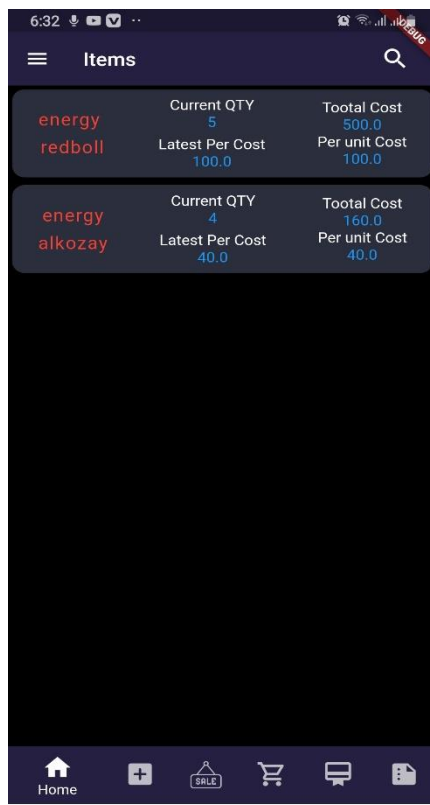
in IMS Mobile App, user can also access (ios and Android phones) From anywhere in the world.

The regular Inventory Management is fully compatible with all different phones iso and Android.

4.3 Home Screen or Items Screen

This page is the First Screen And all access of the application is Home Screen.

This The Main and most use of the App Because of Details about the Stocks and the Searchbar.



4.3 Home Screen Flutter UI Code

These codes are just work on UI of Application and code creating (Home Screen) it means First Screen of the Application.

The First Part of the Codes is a card that Contains all details of Each Items.

And in Second Part we Use that card to show all the items .

In second part we also worked Search bar .

What is a Card ?

Card is a build-in widget in flutter which derives its design from Google's Material Design Library. The functionality of this widget on screen is, that it is a bland space or panel with round corners and a slight elevation on the lower side.

First Part of Code :

```
import 'package:flutter/material.dart';

class ItemCard extends StatelessWidget {
  const ItemCard({Key? key,
    required this.courentPcost,
    required this.totalCost,
    required this.QTY,
    required this.itemName,
    required this.nickName,
    required this.perCost

  }) : super(key: key);

  // final IconData iconData;
  // final Color iconColor;

  //final Color iconBgColor;
  final double courentPcost;
  final double totalCost;
  final int QTY;
  final String itemName;
  final String nickName;
  final double perCost;

  @override
  Widget build(BuildContext context) {

    return Container(

      height: 100,
      child: Card(
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(12)),
        color: Color(0xff2a2e3d),
        child: Row(
          mainAxisAlignment:MainAxisAlignment.spaceAround,

          children: [
```

```

Column(
  crossAxisAlignment: CrossAxisAlignment.center,
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    Text(
      itemName,
      style: TextStyle(
        fontSize: 18,
        letterSpacing: 1,

        color: Colors.red),
    ),
    SizedBox(height:8,),
    Text(
      nickName,
      style: TextStyle(
        fontSize: 18,
        letterSpacing: 1,

        color: Colors.red),
    ),
  ],),

Column(
  crossAxisAlignment: CrossAxisAlignment.center,
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    Text(
      'Current QTY',
      style: TextStyle(
        fontSize: 15,

        color: Colors.white),
    ),
    Text(
      QTY.toString(),
      style: TextStyle(
        fontSize: 15,

        color: Colors.blue),
    ),
    SizedBox(
      height: 5,
    ),
    Text(
      'Latest Per Cost',
      style: TextStyle(
        fontSize: 15,

```